MODULE 01 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

How Python runs your code

Interpreters and compilers, the four parts of a language, keywords, lines, indentation and comments.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 01 of 28

admin@sthithapragna.com

Python checks the **whole file** before it runs a single line. Grammar mistakes stop everything. Other mistakes stop only when they are reached.

Why translation is needed at all

01

Instruction list

The complete set of commands a processor knows. Its alphabet.

02

Machine code

Programs written in that alphabet. Fast, and unreadable to people.

03

Source code

A program in a high-level language, saved in a source file.

04

Translation

A compiler or an interpreter turns source into something runnable.

ON THE EXAM

A processor understands only its own instruction list. Everything else has to be translated first.

The four elements of every language

ELEMENT	WHAT IT IS	IN ENGLISH	IN PYTHON
Alphabet	The symbols you may use	The letters A to Z	Characters in a UTF-8 source file
Lexis	The dictionary of valid words	cat, runs, quickly	Keywords, names, numbers, operators
Syntax	Rules for combining words	'Cat the runs' is broken	print("hi" is missing a bracket
Semantics	Whether a valid phrase makes sense	'The sky ate Tuesday'	10 / 0 is well formed, but meaningless

ON THE EXAM

Syntax errors are caught before anything runs. Semantic errors surface only when that line executes.

admin@sthithapragna.com

Compilation versus interpretation

COMPILATION

Translate once, ship the result

- Source becomes a machine code file, once
- Runs fast, without the compiler present
- Every change means compiling again
- One compiler per hardware platform

INTERPRETATION

Translate every time it runs

- Source is translated each time it runs
- Runs as soon as you finish writing
- The same source runs on any platform
- Slower, and users need the interpreter

THE EXAM ANSWER

Python is an interpreted language. Programs in interpreted languages are often called scripts.

What CPython actually does

01

Source

Your .py file, read as UTF-8 text.

02

Compile

The whole file is parsed and checked. Syntax errors stop here.

03

Bytecode

Compact instructions for a virtual machine, not for the processor.

04

Execute

The Python Virtual Machine runs the bytecode.

WHY IT MATTERS

This is why a syntax error on line 50 stops line 1 from running. And bytecode is not machine code.

admin@sthithapragna.com

.pyc files and the __pycache__ folder

- When you **import** a module, CPython saves its bytecode in a **.pyc** file.
- The .pyc files live in a folder called **__pycache__**, beside the module.
- Names record the interpreter, for example **helper.cpython-312.pyc**.
- The script you run directly is **not** cached. Only imported modules are.

> ON THE EXAM
>
> A .pyc file holds compiled bytecode. Not source code, not machine code, and not an interpreter.

Recognition facts older banks love

- **CPython** is the reference implementation of Python, written in C.
- **Guido van Rossum** created Python and named it after Monty Python's Flying Circus.
- Python 3 is **not** backward compatible with Python 2.
- **IDLE** is the Integrated Development and Learning Environment that ships with Python.
- Source files end in **.py**. Python is free to download and use.

Before running versus while running

SYNTAX ERROR

Found before line 1 runs

```
1  print("first")
2  print("second"
```

Output: nothing at all. SyntaxError: '(' was never closed.

RUNTIME ERROR

Found when the line runs

```
1  print("first")
2  print(1 / 0)
3  print("third")
```

Output: first, then ZeroDivisionError. Line 3 never runs.

THE TIEBREAKER

Syntax errors, including indentation errors, stop the whole file. Runtime errors stop at the line that fails.

Python's 35 keywords

```
False     None      True      and       as        assert    async
await     break     class     continue  def       del       elif
else      except    finally   for       from      global    if
import    in        is        lambda    nonlocal  not       or
pass      raise     return    try       while     with      yield
```

1 Only three start with a capital letter: False, None and True.

1-5 None of these can be a variable or function name. That is a SyntaxError.

Soft keywords match, case, _ and type are not on this list. They are still legal names.

Keywords versus built-in names

KEYWORD

Rejected before running

```
1  True = 1
2  class = "3B"
3  in = [1, 2]
```

Each line is a SyntaxError. Nothing runs.

BUILT-IN NAME

Legal, and quietly harmful

```
1  print = 3
2  print("hi")
```

Line 1 runs. Line 2 fails: TypeError, 'int' object is not callable.

ON THE EXAM

print, input, len and list are names, not keywords. You can reassign them, and then they stop working.

One instruction per line, with three exceptions

```
1  x = 1; y = 2
2  total = x + \
3          y
4  colours = ["red",
5             "green"]
6  print(total, colours)
```

1 A semicolon puts two instructions on one line. Legal, but discouraged.

2-3 A backslash at the end of a line continues the instruction.

4-5 Inside any open bracket, a new line continues automatically.

Two instructions side by side with nothing between them, like print(1) print(2), is a SyntaxError.

Indentation is grammar, not decoration

```
1  score = 82
2  if score >= 70:
3      print("Pass")
4      print("Well done")
5  print("Recorded")
```

2 The colon promises that an indented block follows.

3-4 Same indentation, same block. Both run only if the test is true.

5 Back at the left edge, the block has ended. This line always runs.

PEP 8 recommends four spaces per level. Tabs work, but never mix them with spaces.

Four ways to break indentation

WHAT YOU WROTE	WHAT PYTHON RAISES
A line indented for no reason	IndentationError: unexpected indent
A colon with nothing indented after it	IndentationError: expected an indented block
A block containing only a comment	IndentationError: expected an indented block
Tabs and spaces mixed inconsistently	TabError: inconsistent use of tabs and spaces

THE TRAP

Every one of these is a kind of SyntaxError, so nothing runs. A comment is not a statement: use pass.

What counts as a comment, and what does not

```
1  #!/usr/bin/env python3
2  # A comment runs to the end of the line.
3  total = 10  # This one starts mid-line.
4  label = "# Not a comment, just text"
5  """A string on its own. Python evaluates
6  it and throws it away."""
7  print(total, label)
```

1 The #! line tells Unix-like systems how to run the file. To Python, it is a comment.

3 Code before the # runs. Everything after it is ignored.

4 A # inside quotes is just a character in the string.

5-6 Python has no block comment. A triple-quoted string is a string.

Newer than the syllabus

- Python 3.13 rebuilt the interactive prompt: colour, multi-line editing, and **exit** without brackets.
- Since 3.11, error messages mark the exact failing part of a line.
- Soft keywords arrived: **match**, **case** and **_** in 3.10, **type** in 3.12.
- CPython has an experimental JIT compiler from 3.13. It is off by default.

> ON THE EXAM
>
> None of this changes an exam answer. Python is still interpreted, and there are still 35 keywords.

What to carry into the exam

- Python is **interpreted**. CPython compiles to bytecode, then the PVM runs it.
- Grammar errors, indentation included, stop the file **before line 1** runs.
- 35 keywords. Only **False**, **None** and **True** are capitalised.
- print and input are built-in names, **not** keywords.
- A block holding only a comment is an IndentationError. Use pass.

What happens when this program runs?

```
1  print("Start")
2  total = 10
3  print("Total:", total
4  print("End")
```

A Start is printed, then a SyntaxError is reported

B Start and Total: 10 are printed, then the program fails

C Nothing is printed; the program fails with a SyntaxError

D Start, Total: 10 and End are all printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("Start")
2  total = 10
3  print("Total:", total
4  print("End")
```

SyntaxError: '(' was never closed, reported on line 3.

A Start is printed, then a SyntaxError is reported

Pictures Python running line by line before checking. It compiles the whole file first.

B Start and Total: 10 are printed, then the program fails

True of a runtime error. A missing bracket is a grammar error.

C Nothing is printed; the program fails with a SyntaxError

Line 3 never closes its bracket, so compiling fails and line 1 never runs.

D Start, Total: 10 and End are all printed

Python never skips a broken line. One grammar error stops the file.

THE SIGNAL

A grammar error anywhere means no output at all. Scan brackets, colons and quotes before you trace.

Each line is run on its own. Which two run without an error? (Choose two.)

```
1  in_ = 2
2  True = 1
3  Print = 4
4  class = 3
```

- A Line 1
- B Line 2
- C Line 3
- D Line 4

◆ PAUSE AND DECIDE

Each line is run on its own. Which two run without an error? (Choose two.)

```
1  in_ = 2
2  True = 1
3  Print = 4
4  class = 3
```

A Line 1

in is a keyword, but in_ is a different name. Underscores are allowed.

B Line 2

True is a keyword. SyntaxError: cannot assign to True.

C Line 3

Names are case-sensitive. Print is neither a keyword nor print.

D Line 4

class is a keyword, so this is a SyntaxError.

THE SIGNAL

One character turns a keyword into an ordinary name. Case and underscores both count.

What happens when this program runs?

```
1  score = 72
2  if score >= 70:
3      # passed
4  print("Result recorded")
```

A Result recorded is printed

B Nothing is printed; an IndentationError is raised

C Nothing is printed, and the program ends normally

D Result recorded is printed, then an IndentationError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  score = 72
2  if score >= 70:
3      # passed
4  print("Result recorded")
```

IndentationError: expected an indented block after 'if' statement on line 2.

A. Result recorded is printed
What you would get if line 3 said pass instead of a comment.

B. Nothing is printed; an IndentationError is raised
Comments vanish before the grammar check, so the if block is empty.

C. Nothing is printed, and the program ends normally
An empty block is never silently accepted. It is a grammar error.

D. Result recorded is printed, then an IndentationError
Indentation errors are found before anything runs, not after.

THE SIGNAL

A comment is not a statement. A block that must do nothing needs pass.

> A developer runs main.py, which imports her own module, helper.py. Afterwards, a new folder named __pycache__ has appeared beside helper.py.
>
> Inside it is a single file: helper.cpython-314.pyc.

Which statement about this situation is true?

A The .pyc holds machine code, so helper.py no longer needs an interpreter

B The .pyc holds the source of helper.py, compressed to save space

C main.py is cached there too, because every file that runs is cached

D The .pyc holds compiled bytecode for helper.py, reused by later imports

◆ PAUSE AND DECIDE

Which statement about this situation is true?

A The .pyc holds machine code, so helper.py no longer needs an interpreter

Bytecode targets the Python virtual machine, not the processor. The interpreter is still needed.

B The .pyc holds the source of helper.py, compressed to save space

The source stays in helper.py. The cache holds the result of compiling it.

C main.py is cached there too, because every file that runs is cached

Only imported modules are cached, and the folder holds a single file.

D The .pyc holds compiled bytecode for helper.py, reused by later imports

Importing compiled helper.py to bytecode, cached so the next import skips that step.

THE SIGNAL

A .pyc file contains bytecode. Only imported modules get one, and it never replaces the interpreter.